Spatial Filtering & Convolution

Week 3 — Operating on neighbourhoods

Practical Image Processing

Lecture 3 ▪ 60–70 min

Recap & today

- Wk2: point ops look at one pixel. They can't blur, sharpen, or find edges.
- Today: **spatial filters** — a small **kernel** slides over the image and combines each pixel with its **neighbours**. This is **convolution**.

Why it matters

Smoothing removes noise; sharpening recovers detail; and convolution is the exact operation a CNN learns in Week 13.

Roadmap

Convolution

Smoothing

Sharpening

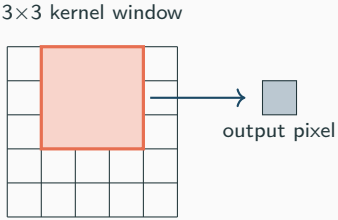
In-class exercise

Deep dive

Wrap-up

Convolution

A kernel slides over the image



For each pixel: multiply the neighbourhood by the kernel, sum, write the result. Slide and repeat.

Convolution, formally

Math

$$g(x,y) = \sum_{i=-k}^{k} \sum_{j=-k}^{k} h(i,j)\, f(x-i,\, y-j)$$

- f = image, h = kernel (a.k.a. filter/mask), g = output.
- The kernel's values *define* the operation: average $\rightarrow$ blur; differences $\rightarrow$ edges.
- Convolution flips the kernel; **correlation** doesn't. For symmetric kernels they're identical.

Two example kernels

Box blur (average):

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Each output = mean of its 3×3 neighbourhood.

Sharpen:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Boosts the centre relative to its neighbours.

Key idea

The kernel *is* the algorithm. Change the numbers, change the effect — nothing else.

Edge handling & normalization

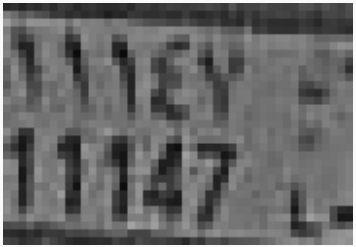
- At the border the kernel hangs off the image — pad with zeros, replicate, or reflect (`borderType`).
- A blur kernel should **sum to 1** (preserve brightness); an edge kernel sums to 0.
- Kernel not normalized → the image gets darker/brighter as a side effect.

✕ Pitfall

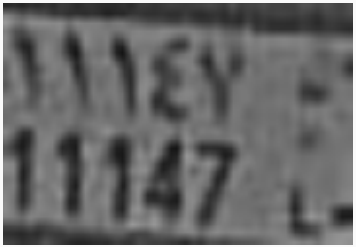
Forgetting to normalize a blur kernel is why a "smoothed" image sometimes comes out too dark.

Smoothing

Box vs Gaussian blur — on a real plate



original



box blur 5×5



Gaussian σ=1.5

Box averages equally (boxy artifacts); Gaussian weights the centre more (smoother, natural).

The Gaussian kernel

Math

$$h(i,j) = \frac{1}{2\pi\sigma^2}\, e^{-(i^2+j^2)/(2\sigma^2)}$$

- σ controls the blur amount; larger σ = more smoothing, wider kernel.
- **Separable**: a 2D Gaussian = a 1D horizontal blur then a 1D vertical blur $\rightarrow$ much faster.
- The go-to smoother: removes noise while preserving structure better than a box.

Why smooth at all?

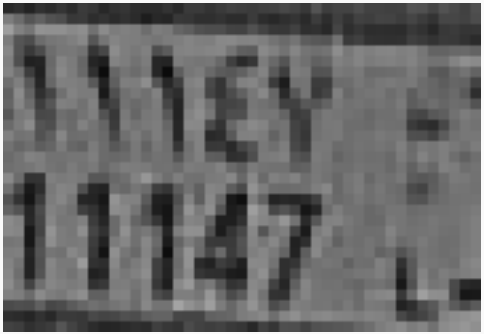
- Remove noise before thresholding/edges (noise creates false edges — Week 5).
- Blur *before* downsampling to avoid aliasing (Week 1's Nyquist).
- Suppress fine texture so large structures dominate (find the page, not the paper grain).

✕ Pitfall

But blur also destroys detail. Too much and the digits smear together — a trade-off you tune.

Sharpening

Sharpening = add back the detail



original



- **Unsharp mask**:
  sharp $=$ img $+ \lambda($img $-$ blur$)$.
- The (img $-$ blur) term *is* the high-frequency detail.
- Adds it back to emphasize edges — the digits' strokes stand out.

Sharpening's catch

- It amplifies *everything* high-frequency — including noise and JPEG blocks.
- Over-sharpening creates **halos** (bright/dark rings) around edges.
- Denoise first, then sharpen gently. Never sharpen a noisy image blindly.

Key idea

Blur and sharpen are opposites, but they're both convolution — just different kernels.

A zoo of kernels

Kernel	Effect
all $1/n$	box blur (average)
Gaussian weights	smooth blur
centre 5, sides -1	sharpen (high-pass)
$[-1, 0, 1]$ rows	horizontal gradient (edges, Week 5)
identity + (id − blur)	unsharp mask

Key idea

You don't memorize kernels — you understand what "average" vs "difference" does, and design the rest.

Bilateral filter: smooth but keep edges

- Gaussian blur smears edges; the **bilateral** filter weights by *both* distance *and* intensity similarity.
- Result: flat regions get smoothed, edges stay crisp — great before thresholding a document.
- Slower, but often the right smoother for DocScan: `cv2.bilateralFilter`.

✔ Good practice

When you need to denoise *without* blurring the digit edges, reach for bilateral (or median).

In-class exercise

In-class exercise (12 min)

Convolve by hand

A 3×3 patch and a kernel:

$$P = \begin{bmatrix} 10 & 10 & 10 \\ 10 & 50 & 10 \\ 10 & 10 & 10 \end{bmatrix} \qquad K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

1. Compute the output for the centre pixel (element-wise multiply, then sum).
2. What does this kernel do to the centre value? What kind of filter is it?
3. Now use the box-blur kernel instead. What is the centre output?

Debrief

- Sharpen: $5(50) - 10 - 10 - 10 - 10 = 250 - 40 = 210$. The centre (50) jumped to 210 — *contrast boosted*.
- It's a **high-pass** filter: it emphasizes the difference between centre and surround.
- Box blur: $(10{\cdot}8 + 50)/9 = 130/9 \approx 14$. The bright centre was *averaged down* — smoothed.

Key idea

Same patch, two kernels, opposite effects. The kernel is everything.

Deep dive

Frequency view: low-pass vs high-pass

- Smoothing keeps **low frequencies** (slow changes), removes high (a **low-pass** filter).
- Sharpening/edges keep **high frequencies** (fast changes) — a **high-pass** filter.
- Week 6 makes this literal with the Fourier transform; today it's intuition.

✓ Good practice

"Blur = low-pass, edges = high-pass" is a lens you'll use for the rest of the course.

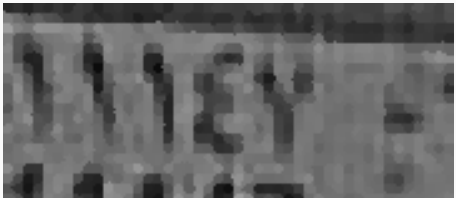
Separability & speed

- A separable 2D kernel = outer product of two 1D kernels; apply them in sequence.
- A $k \times k$ blur costs k^2 multiplies per pixel; separated, only $2k$ — a big win for large k.
- OpenCV does this automatically for Gaussian; know it so you write fast filters.

Nonlinear filter preview: median



salt-and-pepper noise



- A **median** filter takes the median of the neighbourhood — not a weighted sum.
- It kills salt-and-pepper noise while keeping edges sharp (a Gaussian would smear them).
- Nonlinear $\rightarrow$ not convolution, but same "slide a window" idea. Full story in Week 4.

Convolution is the bridge to deep learning

- A CNN layer (Week 13) is exactly this: convolve the image with small kernels.
- The difference: the network *learns* the kernel values from data instead of you designing them.
- Everything you learn about kernels today is what a CNN's first layers do.

Key idea

Classical filters are hand-designed kernels; CNNs are learned kernels. Same operation.

Filtering in code

```
box    = cv2.blur(img, (5, 5))
gauss  = cv2.GaussianBlur(img, (7, 7), 1.5)
median = cv2.medianBlur(img, 3)
k = np.array([[0,-1,0],[-1,5,-1],[0,-1,0]])
sharp  = cv2.filter2D(img, -1, k)          # apply any custom kernel
```

`cv2.filter2D` convolves with any kernel you give it — the general tool.

Wrap-up

What you should now be able to do

- Explain convolution and how a kernel defines an operation.
- Smooth with box/Gaussian; sharpen with unsharp mask; know the trade-offs.
- Handle borders and normalization; exploit separability for speed.
- See the low-pass/high-pass view and the link to CNNs.

This week's deliverable

Due before Week 4 (see `project/`)

On real meter/plate images:

1. Implement a 3×3 convolution *from scratch* (NumPy) and match `cv2.filter2D`.
2. Compare box vs Gaussian blur at several sizes; discuss detail loss vs noise removal.
3. Sharpen a plate with unsharp mask; find where it starts creating halos.
4. Note: what blur helps your DocScan threshold, and what over-blurs the digits?

The kernel is the algorithm.

Blur, sharpen, edges — all the same operation with different numbers.